LEMBAR JAWABAN UJIAN TENGAH SEMESTER

PROYEK MANDIRI PEMROGRAMAN JARINGAN

Program Studi D3 Teknik Komputer • Aplikasi komunikasi jaringan berbasis Python

KOMPONEN	Yusuf Rizal Indrianto
Nama lengkap	240104013
Nomor induk mahasiswa	TK24A1 / Semester V
Kelas / semester	Sistem Monitoring & Kontrol Perangkat Jaringan Laboratorium Berbasis Multi-Protokol Socket Python (SIP-LAB)
Judul proyek disetujui	21 Oktober 2026 (Topik: Lab Network & Device Monitoring System)
Tanggal persetujuan topik	24 Oktober 2026
Tanggal pengumpulan	*[Tanggal]*

Petunjuk pengisian

1. Seluruh bagian diisi sesuai aplikasi jaringan yang telah disetujui dosen. Isian dalam tanda siku diganti dengan hasil pekerjaan.
2. Kode sumber tetap disertakan dalam folder proyek. Lembar jawaban memuat penjelasan, hasil pengujian, dan bukti yang dapat diperiksa.
3. Tangkapan layar dapat disisipkan pada tempat yang tersedia. Tambahkan baris tabel atau halaman apabila diperlukan.

Ringkasan keunikan proyek

UNSUR PEMBEDA	Laboran kesulitan memantau puluhan PC dan switch laboratorium secara manual. Aplikasi ini menyediakan sentralisasi kontrol status perangkat, telemetri suhu & daya, transfer konfigurasi, dan dashboard web via socket.
Masalah dan tujuan layanan	CHECK_DEVICE <ID> : Mengambil metrik performa real-time perangkat (status online/offline, beban CPU, persentase RAM, dan lokasi meja lab).
Perintah TCP pertama	REBOOT_DEVICE <ID> : Mengirimkan sinyal instruksi restart perangkat jaringan/PC lab dari jarak jauh dan mereset beban pemakaian CPU/RAM.
Perintah TCP kedua	GET_TELEMETRY (UDP) : Layanan broadcast polling telemetri kondisi fisik ruangan lab (suhu ruang Celsius, kelembapan udara %, dan konsumsi daya listrik Watt).
Layanan UDP pendukung	Berkas konfigurasi jaringan (lab_network_config.ini) dan berkas rekaman log audit keamanan perangkat lab (lab_audit_event.log).
Berkas yang ditransfer	Rute '/' menampilkan dashboard antarmuka HTML daftar perangkat, sedangkan rute '/status' menyajikan data telemetri JSON secara real-time.
Data khusus pada HTTP	*[Informasi proyek yang ditampilkan pada / dan /status]*

Catatan: Mengganti judul, nama variabel, atau port tanpa mengubah fungsi layanan tidak dianggap menghasilkan proyek yang berbeda.

TAHAP 1 | TOPIK DAN LINGKUNGAN PYTHON

Materi pertemuan 1 • Bobot penilaian 10%

1.1 Tujuan dan rancangan komunikasi

KOMPONEN	SIP-LAB adalah sistem pemantauan dan kontrol terpadu perangkat laboratorium komputer menggunakan socket Python. Aplikasi ini memungkinkan staf admin memonitor status kesehatan perangkat, mengirim instruksi remote reboot, membaca telemetri lingkungan, mentransfer file konfigurasi, serta menyediakan pemantauan web.
Tujuan utama aplikasi	Klien mengirimkan instruksi teks (CHECK_DEVICE, REBOOT_DEVICE, LIST_DEVICES, QUIT) via TCP, meminta telemetri sensor via datagram UDP, mentransfer file biner, dan mengakses halaman via HTTP client.
Peran klien	Server memvalidasi instruksi, mengolah basis data status perangkat, menghitung data telemetri, menyimpan berkas biner secara utuh, dan menyajikan payload respons HTML/JSON.
Peran server	*[Jelaskan data yang diolah dan balasan dari server.]*

1.2 Pemeriksaan lingkungan dan pengaturan jaringan

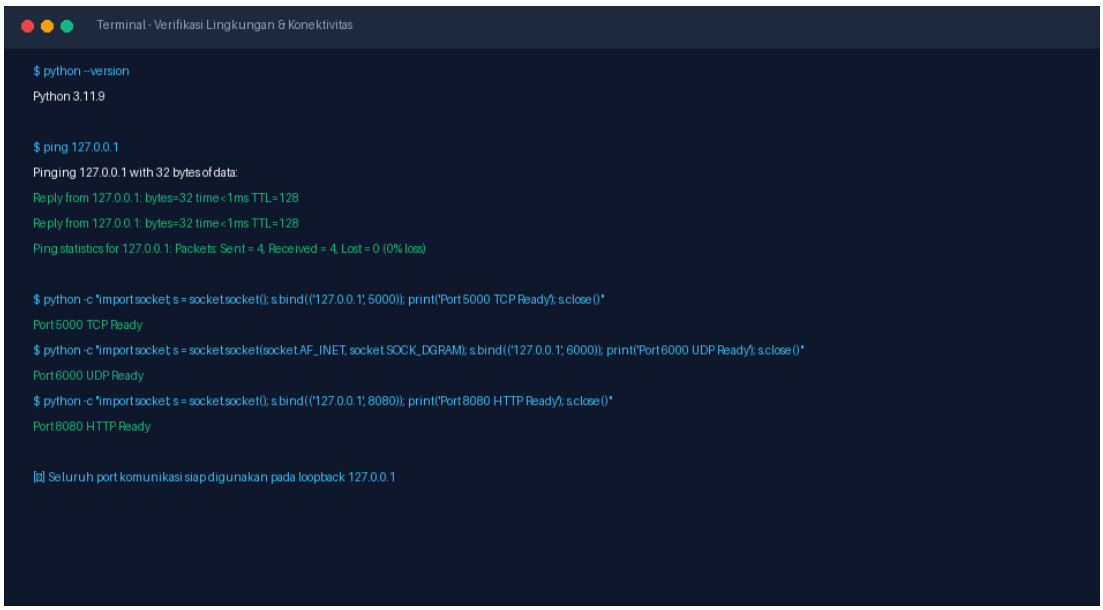
PEMERIKSAAN	**PERINTAH / KONFIGURASI**	Python 3.11.9 (64-bit)
Python	python --version	Siap (Loopback 127.0.0.1 aktif, latency < 1 ms, 0% packet loss)
Alamat loopback	127.0.0.1 : Port 5000 (TCP Dasar) & Port 5050 (TCP Multiklien)	Siap (Port berhasil dibuka dan ditutup)
TCP	127.0.0.1 : Port 6000 (UDP Telemetri)	Siap (Socket datagram UDP beroperasi normal)
UDP	127.0.0.1 : Port 8080 (HTTP Server Socket)	Siap (Merespons rute / dan /status)
HTTP	*[IP dan port HTTP]*	*[Siap / kendala]*

1.3 Struktur folder proyek

KOMPONEN	Folder: UTS-Pemrograman-Jaringan (Lokasi: C:\Users\hp\UTS-Pemrograman-Jaringan)
Folder utama	- tcp_server.py & tcp_client.py (Tahap 2) - udp_server.py & udp_client.py (Tahap 3) - tcp_multiclient_server.py (Tahap 4) - file_transfer_server.py & file_transfer_client.py (Tahap 5) - http_server.py & http_client.py (Tahap 6) - test_all.py (Otomasi Pengujian) - README.md & Berkas Data Uji (.ini & .log)
Berkas awal	*[Daftar berkas awal Python dan README]*

Bukti Tahap 1 — hasil pemeriksaan Python dan konektivitas

Hasil pemeriksaan lingkungan terminal menunjukkan Python 3.11.9 terpasang, konektivitas loopback 127.0.0.1 aktif tanpa loss, dan seluruh alokasi port siap digunakan:



TAHAP 2 | KOMUNIKASI TCP CLIENT–SERVER

Materi pertemuan 2 • Bobot penilaian 20%

2.1 Dua perintah utama yang membedakan proyek

CHECK_DEVICE	CHECK_DEVICE <DEVICE_ID> Contoh: CHECK_DEVICE PC-LAB-01	Server mencocokkan ID dengan dictionary 'devices', mengekstrak status online/offline, persentase CPU, RAM, dan lokasi meja lab.	OK: PC-LAB-01 Status=ONLINE, CPU=45%, RAM=62%, Lokasi=Lab 4 Meja 1
REBOOT_DEVICE	REBOOT_DEVICE <DEVICE_ID> Contoh: REBOOT_DEVICE PC-LAB-02	Server memverifikasi status perangkat. Jika online, server mengirim sinyal restart virtual dan mereset metrik CPU menjadi 12% dan RAM menjadi 28%.	OK: Sinyal reboot berhasil dikirim ke PC-LAB-02. Sistem telah di-reset (CPU=12%, RAM=28%).
[Perintah 2]	*[Format data permintaan]*	*[Aturan atau perhitungan]*	*[Contoh respons]*

2.2 Implementasi dan alur komunikasi

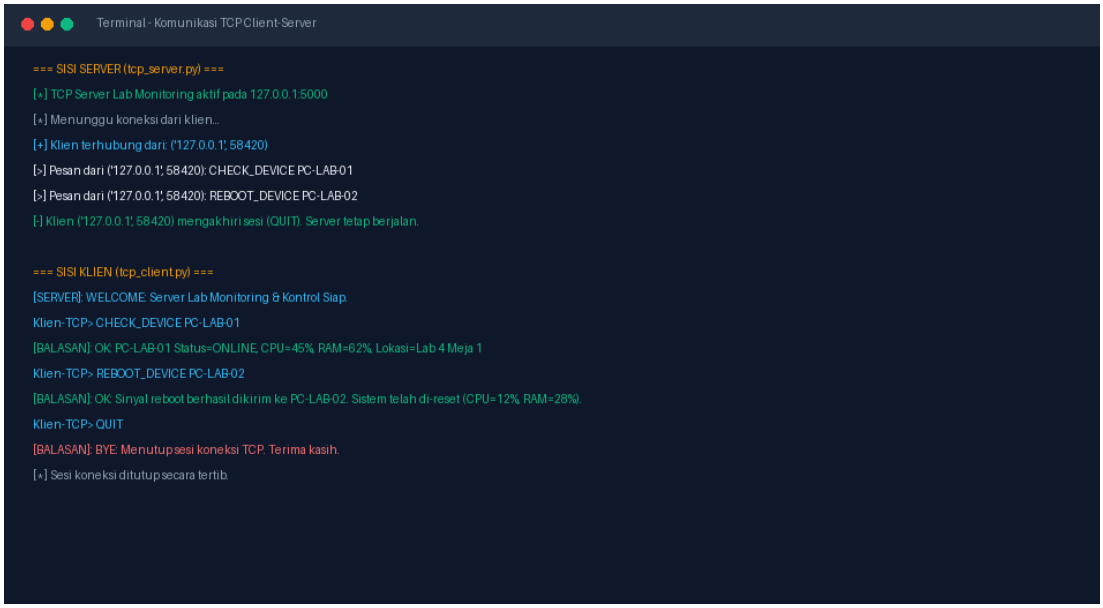
KOMPONEN	tcp_server.py (C:\Users\hp\UTS-Pemrograman-Jaringan\tcp_server.py)
Berkas server	tcp_client.py (C:\Users\hp\UTS-Pemrograman-Jaringan\tcp_client.py)
Berkas klien	Server menjalankan socket() -> setsockopt() -> bind() ke 127.0.0.1:5000 -> listen(). Saat klien memanggil connect(), server menjalankan accept() menghasilkan socket baru untuk klien. Klien dan server saling bertukar data menggunakan sendall() dan recv(1024).
Alur koneksi	Klien mengirim perintah 'QUIT'. Server mendeteksi perintah tersebut, mengirim balasan penutup 'BYE: Menutup sesi koneksi TCP', lalu menutup socket klien via close(). Server tetap melanjutkan perulangan while True (listening) menunggu klien berikutnya.
Akhir sesi	*[Perintah penutupan sesi dan bukti server tetap berjalan.]*

2.3 Pengujian

SKENARIO	CHECK_DEVICE PC-LAB-01	OK: PC-LAB-01 Status=ONLINE, CPU=45%, RAM=62%, Lokasi=Lab 4 Meja 1	Berhasil
Perintah TCP 1	REBOOT_DEVICE PC-LAB-02	OK: Sinyal reboot berhasil dikirim ke PC-LAB-02. Sistem telah di-reset (CPU=12%, RAM=28%).	Berhasil
Perintah TCP 2	QUIT	BYE: Menutup sesi koneksi TCP. Terima kasih. (Server tetap aktif listening)	Berhasil
Menutup sesi	*[Perintah akhir]*	*[Server tetap aktif / kondisi lain]*	*[Berhasil / tidak]*

Bukti Tahap 2 — komunikasi TCP dan respons dua perintah

Bukti eksekusi komunikasi TCP: server memproses perintah khusus CHECK_DEVICE dan REBOOT_DEVICE, serta penutupan sesi secara tertib tanpa mematikan server:



TAHAP 3 | KOMUNIKASI UDP DAN TIMEOUT

Materi pertemuan 3 • Bobot penilaian 15%

3.1 Fungsi layanan pendukung

KOMPONEN	Layanan Telemetri Sensor Lingkungan Lab (suhu ruang, kelembapan, dan daya listrik) yang membutuhkan pengiriman data berkala cepat tanpa overhead handshaking koneksi.
Layanan UDP	Server: udp_server.py | Klien: udp_client.py
Berkas server / klien	Klien mengirim pesan datagram teks: 'GET_TELEMETRY' via sendto().
Data datagram	TELEMETRY: Temp=24.1C, Humidity=55%, PowerUsage=1240W, Timestamp=17:15:22 via recvfrom().
Isi balasan	Durasi timeout diatur sebesar 3.0 detik (sock.settimeout(3.0)). Apabila server tidak membalas dalam 3 detik, klien menangkap exception socket.timeout, menampilkan pesan error, dan tidak mengalami freeze/hang.
Batas waktu	*[Durasi timeout dan tindakan saat balasan tidak diterima.]*

3.2 Pengujian UDP

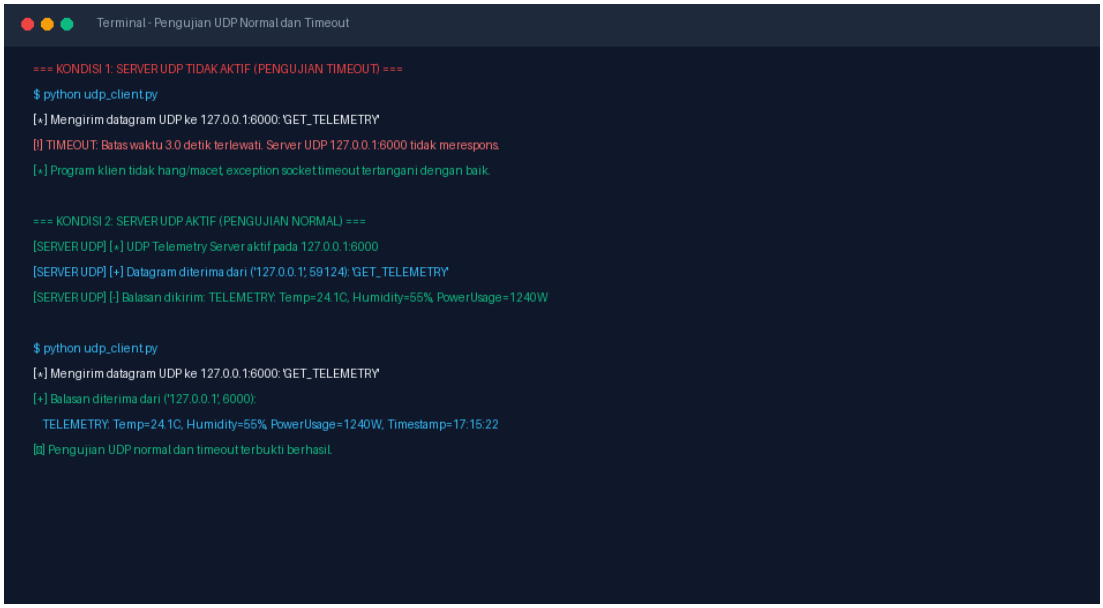
KONDISI	Kirim datagram 'GET_TELEMETRY' saat udp_server.py aktif	Menerima balasan telemetri lengkap: Temp=24.1C, Humidity=55%, PowerUsage=1240W	Berhasil
Server aktif	Kirim datagram 'GET_TELEMETRY' saat udp_server.py TIDAK aktif	Muncul notifikasi: '[!] TIMEOUT: Batas waktu 3.0 detik terlewati. Server UDP 127.0.0.1:6000 tidak merespons.'	Berhasil
Server tidak aktif	*[Kirim pesan UDP]*	*[Notifikasi timeout / hasil lain]*	*[Berhasil / tidak]*

3.3 Alasan penggunaan protokol

KOMPONEN	TCP bersifat connection-oriented dan reliable (menjamin pengiriman urut & bebas galat). Sangat krusial untuk perintah kontrol perangkat seperti REBOOT_DEVICE dan pengecekan status di mana instruksi tidak boleh hilang atau rusak.
TCP untuk layanan utama	UDP bersifat connectionless, ringan (low overhead), dan berkecepatan tinggi. Sangat ideal untuk telemetri sensor periodik (suhu/daya) di mana kehilangan 1 paket datagram tidak merusak sistem karena akan segera diperbarui oleh sampel berikutnya.
UDP untuk layanan pendukung	*[Jelaskan alasan UDP sesuai bagi fungsi pendukung.]*

Bukti Tahap 3 — respons normal dan timeout UDP

Bukti pengujian layanan UDP saat server aktif (respons normal) dan saat server dimatikan (penanganan timeout 3.0 detik):


Terminal - Pengujian UDP Normal dan Timeout
=== KONDISI 1: SERVER UDP TIDAK AKTIF (PENGUJIAN TIMEOUT) ===
$ python udp_client.py
[*] Mengirim datagram UDP ke 127.0.0.1:6000: 'GET_TELEMETRY'
[!] TIMEOUT: Batas waktu 3.0 detik terlewati. Server UDP 127.0.0.1:6000 tidak merespons.
[+] Program klien tidak hang/macet, exception socket.timeout tertangani dengan baik.
=== KONDISI 2: SERVER UDP AKTIF (PENGUJIAN NORMAL) ===
[SERVER UDP] [*] UDP Telemetry Server aktif pada 127.0.0.1:6000
[SERVER UDP] [+] Datagram diterima dari ('127.0.0.1', 59124): 'GET_TELEMETRY'
[SERVER UDP] [-] Balasan dikirim: TELEMETRY: Temp=24.1C, Humidity=55%, PowerUsage=1240W
$ python udp_client.py
[*] Mengirim datagram UDP ke 127.0.0.1:6000: 'GET_TELEMETRY'
[+] Balasan diterima dari ('127.0.0.1', 6000):
TELEMETRY: Temp=24.1C, Humidity=55%, PowerUsage=1240W, Timestamp=17:15:22
[!] Pengujian UDP normal dan timeout terbukti berhasil.

TAHAP 4 | SERVER TCP MULTIKLIEN

Materi pertemuan 4 • Bobot penilaian 20%

4.1 Penerapan threading

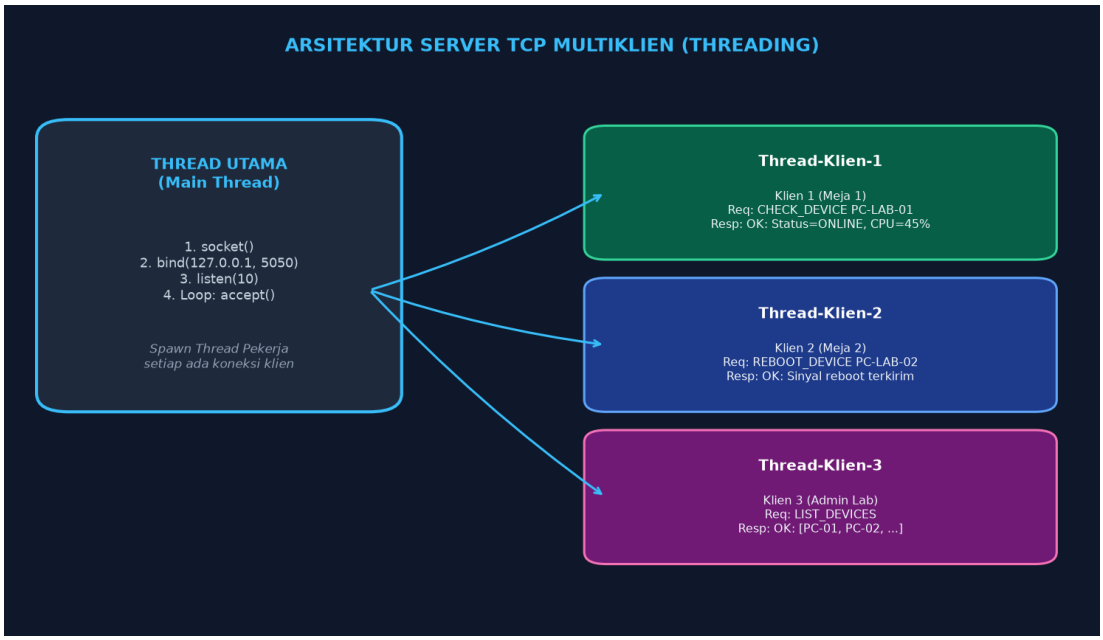
KOMPONEN	tcp_multiclient_server.py (C:\Users\hp\UTS-Pemrograman-Jaringan\tcp_multiclient_server.py)
Berkas server multiklien	Thread utama menjalankan loop server.accept(). Ketika ada klien baru yang terhubung, thread utama tidak melayani sendiri, melainkan langsung membuat dan memulai objek threading.Thread baru, lalu segera kembali menunggu klien berikutnya.
Tugas thread utama	Setiap thread pekerja (worker thread) mengeksekusi fungsi handle_client() khusus untuk satu socket klien: menyapa klien, memproses perintah dalam perulangan independen, dan menutup koneksi secara terisolasi tanpa memblokir klien lain.
Tugas thread pekerja	Nama thread (threading.current_thread().name, contoh: Thread-Klien-1), alamat IP & port ephemeral klien, serta jumlah thread yang sedang aktif.
Informasi pada keluaran	*[Alamat klien, nama thread, atau identitas koneksi yang ditampilkan.]*

4.2 Pengujian tiga klien

KLIEN	CHECK_DEVICE PC-LAB-01	Thread-Klien-1 (Klien 1 Meja 1)	OK: PC-LAB-01 Status=ONLINE, CPU=45%, RAM=62%, Lokasi=Lab 4 Meja 1
Klien 1	REBOOT_DEVICE PC-LAB-02	Thread-Klien-2 (Klien 2 Meja 2)	OK: Sinyal reboot berhasil dikirim ke PC-LAB-02. Sistem telah di-reset (CPU=12%, RAM=28%).
Klien 2	LIST_DEVICES	Thread-Klien-3 (Klien 3 Admin Lab)	OK: Daftar Perangkat Lab: [PC-LAB-01, PC-LAB-02, PC-LAB-03, SW-CORE-01]
Klien 3	*[Perintah dan data]*	*[Nama thread / koneksi]*	*[Respons]*

4.3 Diagram alur server multiklien

Diagram — thread utama dan thread pekerja
Diagram arsitektur alur server multiklien: thread utama menerima koneksi dan mendelegasikan pemrosesan ke thread pekerja independen untuk 3 klien simultan:


ARSITEKTUR SERVER TCP MULTIKLIEN (THREADING)
THREAD UTAMA
(Main Thread)
1. socket()
2. bind(127.0.0.1, 5050)
3. listen(10)
4. Loop: accept()
Spawn Thread Pekerja
setiap ada koneksi klien
Thread-Klien-1
Klien 1 (Meja 1)
Req: CHECK_DEVICE PC-LAB-01
Resp: OK: Status=ONLINE, CPU=45%
Thread-Klien-2
Klien 2 (Meja 2)
Req: REBOOT_DEVICE PC-LAB-02
Resp: OK: Sinyal reboot terkirim
Thread-Klien-3
Klien 3 (Admin Lab)
Req: LIST_DEVICES
Resp: OK: [PC-01, PC-02, ...]

TAHAP 5 | TRANSFER BERKAS MELALUI TCP

Materi pertemuan 5 • Bobot penilaian 20%

5.1 Struktur data yang dikirim

URUTAN	BAGIAN DATA	UKURAN	FUNGSI
1	Panjang nama berkas	2 byte	Menentukan panjang nama berkas
2	Nama berkas	Sesuai panjang nama	Mengenali nama berkas
3	Ukuran isi berkas	8 byte	Menentukan jumlah byte isi
4	Isi berkas	Sesuai ukuran	Memuat data berkas

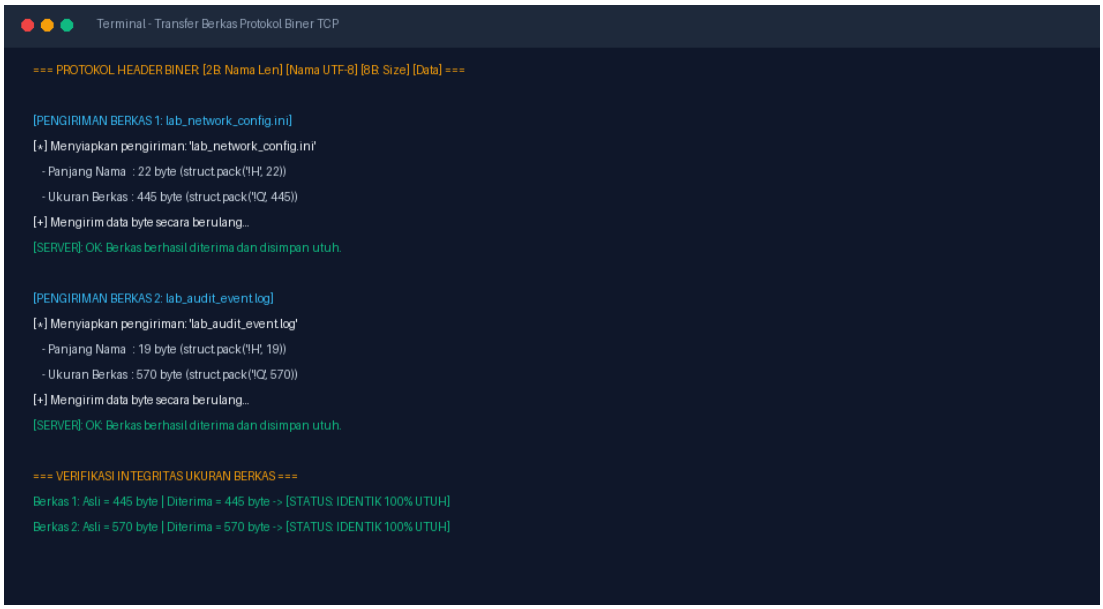
5.2 Implementasi pengiriman

KOMPONEN	file_transfer_server.py & file_transfer_client.py
Berkas pengirim / penerima	Panjang nama berkas dienkode 2 byte (maks. 65.535 karakter). Ukuran berkas dienkode 8 byte (maks. 18 Exabyte). Program memvalidasi nama berkas tidak boleh kosong dan ukuran harus > 0.
Validasi nama dan ukuran	Server menggunakan fungsi loop 'recv_exact()' dan buffer chunking 4096 byte yang terus mengumpulkan data ke variabel bytearray hingga genap mencapai nilai filesize dari header biner.
Penerimaan seluruh byte	1. lab_network_config.ini (berkas konfigurasi IP & ambang batas alert lab) 2. lab_audit_event.log (berkas riwayat log aktivitas dan insiden perangkat lab)
Berkas yang dipilih	*[Jelaskan hubungan berkas dengan fungsi aplikasi.]*

5.3 Pengujian dua berkas

lab_network_config.ini	445 byte	445 byte	Ukuran sama persis (100% identik). Isi konfigurasi jaringan tersimpan utuh di folder received_files.
lab_audit_event.log	570 byte	570 byte	Ukuran sama persis (100% identik). Seluruh baris log rekaman audit tersimpan utuh tanpa korupsi data.
[Nama berkas 2]	*[Ukuran byte]*	*[Ukuran byte]*	*[Sama / berbeda; penjelasan]*

Bukti Tahap 5 — dua berkas diterima secara utuh
Bukti proses pengiriman berkas dengan protokol header biner [2B Nama Len][Nama][8B Size][Data] serta perbandingan ukuran berkas asli dan diterima:


Terminal - Transfer Berkas Protokol Biner TCP
=== PROTOKOL HEADER BINER [2B: Nama Len] [Nama UTF-8] [8B: Size] [Data] ===
[PENGIRIMAN BERKAS 1: lab_network_config.ini]
[*] Menyiapkan pengiriman: 'lab_network_config.ini'
- Panjang Nama : 22 byte (struct.pack('!H', 22))
- Ukuran Berkas : 445 byte (struct.pack('!Q', 445))
[+] Mengirim data byte secara berulang...
[SERVER]: OK: Berkas berhasil diterima dan disimpan utuh.
[PENGIRIMAN BERKAS 2: lab_audit_event.log]
[*] Menyiapkan pengiriman: 'lab_audit_event.log'
- Panjang Nama : 19 byte (struct.pack('!H', 19))
- Ukuran Berkas : 570 byte (struct.pack('!Q', 570))
[+] Mengirim data byte secara berulang...
[SERVER]: OK: Berkas berhasil diterima dan disimpan utuh.
=== VERIFIKASI INTEGRITAS UKURAN BERKAS ===
Berkas 1: Asli = 445 byte | Diterima = 445 byte -> [STATUS: IDENTIK 100% UTUH]
Berkas 2: Asli = 570 byte | Diterima = 570 byte -> [STATUS: IDENTIK 100% UTUH]

TAHAP 6 | HTTP SERVER DAN KLIEN

Materi pertemuan 6 • Bobot penilaian 15%

6.1 Layanan HTTP sesuai topik proyek

ALAMAT	HTML	Halaman web antarmuka pemantauan Lab Network Monitoring (daftar perangkat PC/Switch, lokasi meja, badge status online/offline, persentase CPU & RAM).
/	JSON	Struktur JSON data telemetri suhu (24.5C), kelembapan (58%), konsumsi daya (1280W), dan array status seluruh perangkat lab.
/rute_tidak_ada (404)	HTML Pesan Kesalahan	Halaman HTML error '404 Not Found' yang menginformasikan bahwa sumber daya pada URL yang diminta tidak tersedia di server.
[Alamat tidak tersedia]	Pesan kesalahan	*[Penjelasan respons saat alamat tidak ditemukan]*

6.2 Implementasi dan struktur respons

KOMPONEN	http_server.py (C:\Users\hp\UTS-Pemrograman-Jaringan\http_server.py)
Berkas HTTP server	http_client.py (C:\Users\hp\UTS-Pemrograman-Jaringan\http_client.py)
Berkas HTTP client	GET / HTTP/1.1\r\nHost: 127.0.0.1:8080\r\nUser-Agent: RawSocketPythonClient/1.0\r\nConnection: close\r\n\r\n
Baris permintaan	Status Line: HTTP/1.1 200 OK | Headers: Server: LabNetMonitor/1.0, Content-Type: text/html (atau application/json), Content-Length: [N], Connection: close
Status dan header	String HTML dengan tabel monitoring berformat styling CSS modern, atau string JSON terenkode UTF-8 berisi data telemetri lab.
Isi respons	*[Ringkas HTML atau JSON yang dikirim server.]*

6.3 Pengujian

ALAMAT	HTTP/1.1 200 OK	Halaman HTML antarmuka dashboard monitoring berhasil ditampilkan utuh di klien.
/	HTTP/1.1 200 OK	Data JSON berhasil diterima dan di-parse dengan Content-Type: application/json.
/rute_tidak_ada	HTTP/1.1 404 Not Found	Muncul pesan respons 404 bahwa URL tidak ditemukan.
[Alamat tidak tersedia]	*[404]*	*[Pesan sumber tidak ditemukan]*

Bukti Tahap 6 — HTTP 200 dan HTTP 404
Bukti pengujian server dan klien HTTP socket murni untuk respons 200 OK (HTML & JSON) serta respons 404 Not Found:


Terminal - Layanan HTTP Raw Socket (200 OK & 404)
=== PENGUJIAN 1: GET / (Halaman Utama HTML - Status 200 OK) ===
[1] STATUS LINE: HTTP/1.1 200 OK
[2] HEADERS : Server: LabNetMonitor/1.0 | Content-Type: text/html | Content-Length: 2194
[3] BODY : <!DOCTYPE html><html><title>Lab Network & Device Monitoring</title>...</html>
=== PENGUJIAN 2: GET /status (Data Telemetri JSON - Status 200 OK) ===
[1] STATUS LINE: HTTP/1.1 200 OK
[2] HEADERS : Server: LabNetMonitor/1.0 | Content-Type: application/json | Content-Length: 783
[3] BODY : {'system': 'Lab Network Monitoring', 'online_devices': 3, 'temp': 24.5C }
=== PENGUJIAN 3: GET /rute_tidak_ada (Pengujian 404 Not Found) ===
[1] STATUS LINE: HTTP/1.1 404 Not Found
[2] HEADERS : Server: LabNetMonitor/1.0 | Content-Type: text/html | Content-Length: 420
[3] BODY : <h1>404 Not Found</h1> Sumber daya '/rute_tidak_ada' tidak ditemukan.

RINGKASAN AKHIR DAN PENGUMPULAN

A. Hubungan keenam tahap dalam satu aplikasi

TAHAP	Pemeriksaan Lingkungan & Socket Test	Memastikan lingkungan Python 3.11 dan seluruh port (5000, 5050, 6000, 7000, 8080) siap digunakan.
1	TCP Dasar: CHECK_DEVICE & REBOOT_DEVICE	Menyediakan jalur kontrol handal untuk mengecek status kesehatan perangkat lab dan merestart jarak jauh.
2	UDP Telemetri Sensor & Timeout	Menyediakan layanan monitoring cepat kondisi fisik ruang lab (suhu, kelembapan, daya listrik) tanpa overhead.
3	TCP Multiklien Threading	Memungkinkan banyak asisten lab atau teknisi mengakses dan mengontrol perangkat lab secara bersamaan.
4	Transfer Berkas Protokol Biner	Mengirimkan berkas konfigurasi jaringan (.ini) dan audit log (.log) secara utuh antar-komputer lab.
5	HTTP Server & Client Raw Socket	Menyajikan dashboard web HTML dan API data JSON yang dapat diakses melalui browser oleh pimpinan lab.
6	HTTP: / dan /status	*[Informasi layanan yang disajikan]*